One buy and its fee

Date: 14 October 2026. Task: L001b, portfolio accounting equations. Sidiq review status: Not yet reviewed.

Research question and hypothesis

How does one affordable buy change cash, shares and marked wealth when its fill price equals the unchanged mark? Prediction, recorded before implementation: the purchase moves money from cash into holdings; the fee reduces wealth. Wealth after the buy equals wealth before minus the fee. Yesterday's L001a established wealth = cash + shares * mark; today adds one known purchase.

This is a synthetic accounting fixture, not a trading strategy. There is no historical period, market download, signal, fitting or parameter search. The protocol is a hand calculation, a fixed C++17 program, an independent Python Decimal ledger, two exact executable runs, two identical JSON replays and deliberate incorrect-code controls. Metrics are exact amounts and reconciliation identities, not returns, risk-adjusted performance or statistical significance.

Beginner explanation and vocabulary

Cash is money available in the simplified account. Position is the number of shares owned. The mark values each share; the fill price is the price paid for a purchased share. Today they are both 50 money units per share and remain fixed. A fee is a separate amount paid once for this buy. Notional means the price of the shares alone, excluding the fee.

Start with 800 money units and four shares. Buying two more shares does not destroy the 100 paid for them: cash falls by 100 while holdings rise by 100. Paying the separate fee of 3 removes another 3 from cash without adding a share. Therefore total marked wealth falls by 3. There is no new deposit, borrowing or outside funding. The seller and fee recipient are outside this account; its wealth is not conserved when money leaves as a fee.

Symbol	Meaning	Units / fixed value
C	Cash before purchase	Money units: 800
q	Shares already owned	Shares: 4
n	Shares bought	Shares: 2
p	Fill price and unchanged mark	Money units per share: 50
f	Fee paid once	Money units: 3

All money is in whole arbitrary units, not pounds, dollars or fractional currency. Shares are whole counts. Known constants are nonnegative, price and buy quantity are positive, and the cash requirement including the fee is affordable. No input parser or general trading API is offered.

Derivation, units and hand example

Notional N = n * p = 2 * 50 = 100. Shares times money per share gives money. The total cash debit is N + f = 103, so the affordability condition is C >= N + f. Checking only N would ignore a real cash obligation in this model.

Cash after C_after = C - N - f = 800 - 100 - 3 = 697. Shares after q_after = q + n = 4 + 2 = 6. Holdings before H_before = q * p = 200; holdings after H_after = (q + n) * p = 300. Wealth before W_before = C + q * p = 1000. Wealth after W_after = C_after + q_after * p = 697 + 300 = 997.

Substitute the cash and shares equations before simplifying: W_after = (C - n*p - f) + (q + n)*p = C + q*p - f = W_before - f. The two n*p terms cancel because the acquired shares are valued at the same price paid. The cash expenditure on shares is a change in asset composition, not itself a wealth loss.

Reconciliation	Calculation	Result
Cash used	800 - 697	103 = 100 + 3
Holdings gained	300 - 200	100
Shares acquired	6 - 4	2
Wealth change	997 - 1000	-3
Restore fee to reconcile	997 + 3	1000

A zero-fee hand counterfactual would leave cash 700, holdings 300 and wealth 1000. It is a comparison of assumptions, not a second compiled case or evidence that real trading is free. If someone reports a loss of 103, they have counted the cash debit but forgotten the new holdings. If someone reports no loss at all, they have likely omitted the fee.

The small C++ calculation, line by line

In cpp/projects/portfolio_accounting/buy_equations.cpp, #include <iostream> enables printed output; int main() is the program entry point. The opening and closing braces enclose its statements. Each const int declaration gives a whole number a name that cannot later be reassigned. The first five declarations fix cash_before, shares_before, price, buy_shares and fee.

wealth_before applies yesterday's equation. notional multiplies buy_shares by price. cash_after subtracts that notional and then the fee once. shares_after adds only the purchased share count. holdings_after multiplies the new count by price. wealth_after adds cash and holdings. The four std::cout statements print the before, order, after and reconciliation lines. The << operator sends each value to the stream; '\n' ends a line. Semicolons end statements.

No struct, reusable buy function or mutable account is introduced today. All fixed arithmetic is between -3 and 1000, within even the minimum int range. This bounded safety argument does not make int safe for arbitrary money amounts; range and representation need explicit policy.

Implementation, checks and observed results

Run make cpp-buy-equations to compile with C++17 and warnings treated as errors, then run the independent verifier. Run .venv/bin/python scripts/verify_buy_equations.py for its JSON report. The hand-authored buy_expected.txt fixes all four output lines before the first execution. The archived report is experiments/portfolio_accounting/buy-equations.json.

The Python reference debits 50 twice and increases the share count once for each debit. It then debits 3 once. Holdings are valued by adding 50 for each share, rather than using the C++ multiplication. Decimal arithmetic and hand-known totals check cash, holdings, wealth and both reconciliation identities. The reference checks the fixed input assumptions and affordability.

Observed C++ cash=697, shares=6, holdings=300 and wealth=997 exactly match. Every output byte, empty stderr and exit status 0 match twice per verifier invocation; two JSON replays are identical. A scratch program omitting the fee compiles but fails the reference comparison. Another scratch program forgetting the extra shares also compiles but fails. Both verifier failures exit 1 for the intended mismatch. Production source remains unchanged by these controls. An actual Python -O run is rejected by the explicit guard, so disabled assertions cannot silently pass the check.

Full make check passes: 608 Python tests; Ruff lint/format; strict package typing; 66 savings and 30 parser checks; 75 parser boundary comparisons; four earlier golden outputs; fractional failure controls; eight readiness traces twice; three marked-wealth snapshots twice; and today's one-buy fixture twice. Dependency consistency passes. These counts are not a measured coverage percentage. The new fixture passed first run. The negative controls are deliberately wrong, not discovered bugs.

Costs, failures and limits

The fee is a chosen constant, not a broker quote or an estimate. Spread, impact, taxes, borrowing, interest, dividends, multiple currencies and settlement delays are omitted. The simplified buy is filled immediately and completely at the mark. This does not establish that such a fill is available. A fill above the mark would make the immediate wealth reduction larger than the explicit fee; the cancellation in today's proof depends on equal prices and an unchanged mark.

An affordable notional does not imply an affordable total debit. With cash 102, this order would need 103 and must be rejected in the future validated interface. That rejection and preservation of state are planned L001d/e work, not capabilities demonstrated by today's fixed program. A fee paid twice would also misstate wealth. Do not put the fee into marked holdings to hide it: valuation is share count times mark, regardless of purchase costs or tax cost-basis conventions.

The two implementations share the fixture assumptions; agreement cannot prove market realism. No timing model, past-decision invariance, sell, short position, borrowing or general overflow handling is implemented. Q004a/Q005b continue to gate real-data work. No empirical profit is claimed.

Primary sources consulted on October 14

Investor.gov, Investor Bulletin: Brokers' Miscellaneous Fees, December 15, 2014. The bulletin separates the security purchase amount from fees and explains that costs reduce returns. Our amounts and equal-price accounting model are assumptions derived here, not market evidence. https://www.investor.gov/introduction-investing/general-resources/news-alerts/alerts-bulletins/investor-bulletins-82

Microsoft, Built-in types (C++). Integral types represent whole numbers over a bounded range; our small fixed integers avoid fractional-money and large-value design questions today. https://learn.microsoft.com/en-us/cpp/cpp/fundamental-types-cpp?view=msvc-170

Python, subprocess documentation. Captured stdout/stderr, return codes and a timeout support the exact executable checks; the accounting oracle itself is our independent calculation. https://docs.python.org/3/library/subprocess.html#subprocess.run

Interview explanation

I reconcile a buy by subtracting quantity times fill price plus the fee from cash, then adding the quantity to holdings. At an unchanged mark equal to the fill, the purchase notional cancels from total wealth and only the fee reduces wealth. I check cash and shares separately; wealth alone can hide offsetting errors. This validates a fixed accounting example, not a strategy.

Progress, publication and next task

L001b's equations and fixed demonstration are complete. Next is L001c: introduce a single position struct and explain each field before L001d adds a validated buy function. Independent boundaries, reproduction, L002 timing and future-price invariance remain unfinished. C003b/C004/C005 retain their prerequisites; later strategy dates remain conditional. No answer is needed for the next synthetic lesson. Personal review remains Not yet reviewed; real-data access still needs evidence.

Today's shared London-date draw is 3. No earlier commits were made today and all previous pending packages already have integration receipts. One coherent lesson is ready; no extra lesson or artificial split fills the other slots. Commit/push outcomes are verified separately. The daily schedule remains 08:00 Europe/London through December 31, then new research stops pending scope.

Check your understanding

1. Why is the cash debit 103 while the wealth decrease is only 3?

2. Why must affordability include the fee? Can cash 102 fund this buy?

3. Does a zero fee prove a real purchase cannot reduce marked wealth?

4. What is wrong with counting the fee as additional marked holdings?

Suggested answers

1. The 100 paid for shares becomes holdings worth 100 at the unchanged equal mark; only the fee leaves the account without adding a marked asset. Cash is 697; holdings are 300; wealth is 997.

2. The total obligation is 2*50+3=103, so 102 is insufficient. A future validated buy must reject it without changing cash or shares. Today's fixed program does not implement that interface.

3. No. Today's cancellation also assumes fill equals mark, full execution and no other costs. Different execution/valuation prices can change wealth even with no separately charged fee.

4. Marked holdings are six shares times 50 = 300. Adding the fee would overstate them and erase the observed wealth cost. Purchase cost basis and current marked valuation are different concepts.